K.Shamela

Mini-Project -*3*

Project Title : *Content Monetization Modeler*

Course : *Data science*

Batch : *DS-C-WD-E-B144*

Content Monetization Modeler

Problem Statement:

YouTube creators often find it difficult to estimate how much revenue a video might generate from advertisements. While some metrics such as views and engagement provide clues, the relationship between these metrics and revenue is not always straightforward.

The goal of this project is to develop a machine learning model that predicts the expected ad revenue of a video based on its performance characteristics. By analyzing these factors, the project also aims to provide insights into what drives higher revenue for content creators.

Introduction

With the rapid growth of online video platforms, content creators rely heavily on advertisement revenue generated from their videos. However, estimating the potential ad revenue of a video based on its performance metrics can be challenging. Understanding which factors influence revenue can help creators optimize their content strategy and maximize earnings.

This project focuses on building a machine learning regression model that predicts YouTube advertisement revenue using various video performance metrics and contextual features. The project also includes an interactive web application that allows users to input video metrics and obtain predicted ad revenue.

The main objective of this project is to analyze which factors significantly impact video monetization and to create a predictive system that estimates revenue based on those factors.

Dataset Description:

The dataset used in this project contains information about YouTube video performance and monetization metrics. Each row represents a video and includes several features related to engagement, audience behaviour, and contextual information. Each row in the dataset represents performance metrics for a specific video on a specific day.

Key Features

- **Views** – Total number of times the video has been watched.
- **Likes** – Number of users who liked the video.
- **Comments** – Number of comments posted on the video.
- **Watch Time (Minutes)** – Total viewing time accumulated by the video.
- **Video Length (Minutes)** – Duration of the video.
- **Subscribers** – Total number of subscribers of the channel.
- **Category** – Type of content such as Entertainment, Gaming, Music, etc.
- **Device** – Primary device used by viewers (Mobile, TV, Tablet).
- **Country** – Top geographic location of the viewers.

Target Variable

- **Ad Revenue** – Estimated revenue generated from advertisements displayed on the video.

Data Preprocessing

Before training the machine learning models, several preprocessing steps were applied to ensure data quality and consistency.

Handling Missing Values

Missing numerical values were handled using mean imputation. Replacing missing values with the mean ensures that the dataset remains complete while preserving the overall distribution of the data.

```
df["likes"] = df["likes"].fillna(df["likes"].mean())
df["comments"] = df["comments"].fillna(df["comments"].mean())
df["watch_time_minutes"] = df["watch_time_minutes"].fillna(df["watch_time_minutes"].mean())
```

df.isnull().sum() using this we check the null values and get 0 null values in all columns.

Remove the Duplicates:

Duplicates in a dataset can cause bias in analysis and incorrect model predictions. Removing them ensures data consistency and accuracy.

```
df.duplicated().sum()
✓ 0.1s

np.int64(2400)

df = df.drop_duplicates()
✓ 0.1s

df.duplicated().sum()
✓ 0.1s

np.int64(0)
```

Encoding Categorical Variables

Categorical variables such as video category, device type, and country were converted into numerical format using **One-Hot Encoding**. This technique creates binary columns for each category so that machine learning algorithms can process the data effectively.

```
df = pd.get_dummies(df,columns=["category","device","country"],dtype='int',drop_first=True)
✓ 0.0s
```

Feature Engineering

A new feature called **engagement_rate** was created to measure how actively viewers interact with the content.

Formula:

engagement_rate = (likes + comments) / views

```
df['engagement_rate'] = (df["likes"] + df["comments"])/df["views"]
```
✓ 0.0s

This feature helps capture audience engagement relative to the total number of views and provides a better representation of viewer interaction.

```
df.drop("video_id",axis=1,inplace=True)
```
✓ 0.0s

It does not help the model learn patterns, so we remove it.

Feature Scaling

Numerical features were standardized using **StandardScaler** to normalize the scale of the variables. Scaling ensures that large numerical values such as views do not dominate smaller features during model training.

```
#Feature scaling
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Machine Learning Models

Multiple regression algorithms were trained and evaluated to determine which model performs best for revenue prediction.

Models Implemented

- Linear Regression
- Decision Tree Regressor
- Random Forest Regressor
- Gradient Boosting Regressor
- Support Vector Regressor (SVR)

Linear Regression:

Linear Regression is a simple and widely used algorithm that models the relationship between input features and the target variable using a straight-line equation. It assumes a linear relationship between variables and is easy to interpret, making it useful for understanding how different features influence revenue.

Decision Tree Regressor:

Decision Tree Regressor predicts values by splitting the dataset into smaller subsets based on feature conditions. It creates a tree-like structure where each branch represents a decision rule. This model can capture non-linear relationships and interactions between features.

Random Forest Regressor:

Random Forest is an ensemble learning method that combines multiple decision trees to improve prediction accuracy. Each tree is trained on a random subset of the data, and the final prediction is obtained by averaging the outputs of all trees. This approach reduces overfitting and improves model stability.

Gradient Boosting Regressor:

Gradient Boosting builds models sequentially, where each new model attempts to correct the errors made by the previous one. By combining many weak learners (usually decision trees), the model gradually improves prediction performance and can capture complex patterns in the data.

Support Vector Regressor (SVR):

Support Vector Regressor is based on the concept of support vector machines. It attempts to find a function that best fits the data while keeping prediction errors within a specified margin. SVR is effective for handling non-linear relationships and works well with high-dimensional datasets.

These models were compared using evaluation metrics to determine the most suitable model for the dataset

Model Evaluation

The models were evaluated using three primary regression metrics:

- **R^2 Score** (Coefficient of Determination) – Measures how well the model explains the variance in the target variable.
- **MAE (Mean Absolute Error)** – Represents the average magnitude of prediction errors.
- **RMSE (Root Mean Squared Error)** – Penalizes larger prediction errors more heavily.

Model Performance Results

Model	R2 Score	MAE	RMSE
Linear Regression	0.952574	3.118559	13.479979
Gradient Boosting	0.952275	3.620905	13.522416
Random Forest	0.949856	3.586600	13.860990
SVR	0.946627	5.938412	14.300227
Decision Tree	0.898562	5.446318	19.714371

Best Model Selection

Among all models, ***Linear Regression*** provided the best balance between accuracy and simplicity. It achieved the highest R^2 score while maintaining relatively low MAE and RMSE values.

Therefore, Linear Regression was selected as the final model for deployment in the application.

Model Interpretation and Insights

Analyzing the model helped identify which features most strongly influence advertisement revenue.

Key Insights

Views

The number of views has the strongest impact on ad revenue. Higher view counts increase the number of ad impressions, which directly contributes to increased earnings.

Engagement Rate

Videos with higher engagement rates tend to generate higher revenue. Engagement indicates that viewers interact with the content, which may increase watch time and advertiser interest.

Watch Time

Longer watch times indicate that viewers stay engaged with the video for extended periods. Higher watch time increases the likelihood that advertisements will be displayed during the viewing session.

Subscribers

Channels with a larger subscriber base typically attract more viewers, leading to higher ad revenue.

Content Category

Certain categories of content perform better depending on audience demand and advertising competition.

Streamlit Application

To demonstrate the practical application of the trained model, an interactive web application was developed using Streamlit. The application interface was implemented in a Python file named app.py, which loads the trained machine learning model and allows users to interact with it through a simple web interface.

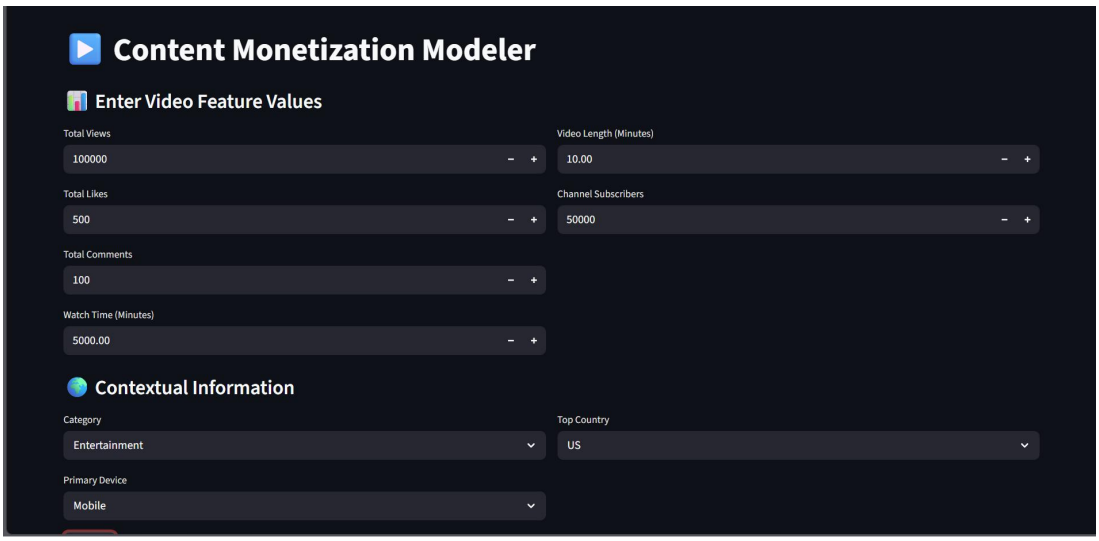
After training the model, it was saved using the joblib library. Joblib is commonly used to serialize and store trained machine learning models so they can be reused later without retraining. The saved model file is then loaded inside app.py when the application starts.

```python
import joblib

# We will save the Linear Regression model (which is models["Linear Regression"])
best_model = models["Linear Regression"]

# Save the model and the scaler
joblib.dump(best_model, 'linear_regression_model.pkl')
joblib.dump(scaler, 'scaler.pkl')

# Save the list of feature columns so our app knows the exact inputs it needs
joblib.dump(list(X.columns), 'model_columns.pkl')

print("Model and Scaler successfully saved!")
```

✓ 0.1s

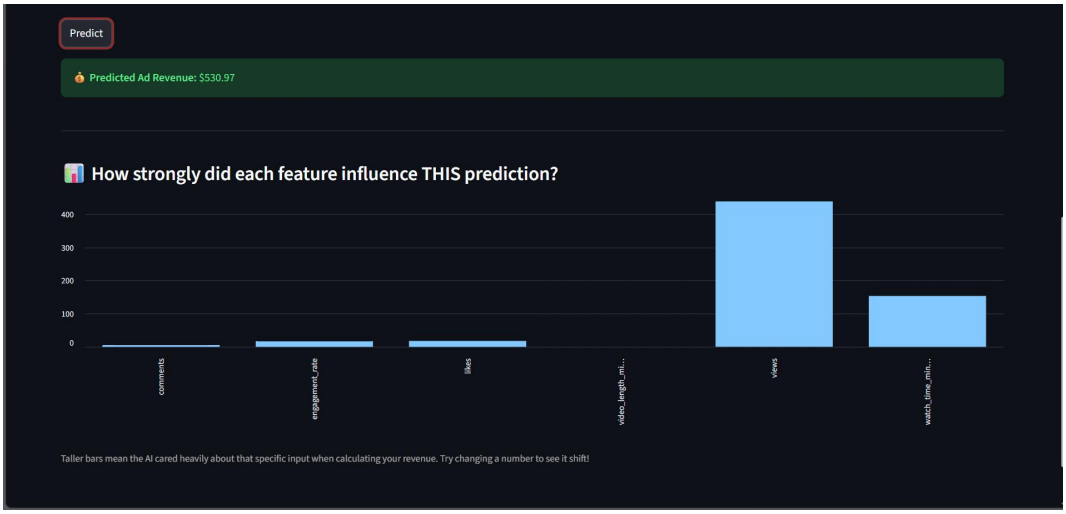
```
Model and Scaler successfully saved!
```

Within the Streamlit application, users can input video performance metrics such as views, likes, comments, watch time, video length, and subscriber count. They can also select categorical attributes such as video category, device type, and viewer country. These inputs are processed in the same way as the training data to ensure consistency.



Once the user provides the inputs, the application sends the processed data to the loaded model, which generates a predicted ad revenue value. The prediction is then displayed instantly on the interface.

In addition to generating predictions, the application also includes a bar chart visualization that helps users understand which features influence revenue the most. The bar chart displays the importance or contribution of key features such as views, engagement rate, watch time, and subscribers. This visualization allows users to easily interpret the model's behavior and identify which factors have the strongest impact on predicted revenue.



The bar chart is generated dynamically within app.py using visualization libraries such as Matplotlib or Plotly and is displayed directly in the Streamlit interface. By combining prediction results with visual insights, the application not only predicts revenue but also helps users better understand the factors that drive monetization performance.

This setup demonstrates how a trained machine learning model can be integrated into a user-friendly application, allowing users to experiment with different inputs and observe how changes in video performance metrics influence predicted revenue while also visualizing feature influence through the bar chart.

Conclusion

This project successfully developed a machine learning model capable of predicting YouTube advertisement revenue using video performance data.

The results indicate that metrics such as views, engagement rate, and watch time play a crucial role in determining revenue. The developed model demonstrates strong predictive performance and provides valuable insights into the factors influencing monetization.

Additionally, the interactive application allows users to easily test the model and explore predictions in real time, making the project both practical and informative.

Future Improvements

Several enhancements could further improve the project:

- Incorporating larger and more diverse datasets
- Adding additional features such as audience demographics
- Using advanced machine learning models
- Deploying the application as a cloud-based service for broader accessibility

These improvements could increase the accuracy and usability of the system in real-world scenarios.